

Hashiko

Distributed File Integrity Monitoring System

Table of Contents

Document Control	3
List of Abbreviations	4
Abstract	5
Student Background	5
Project Summary	6
Problem Statement and Background	7
Inventiveness	8
Complexity	8
Technical Challenges	9
Methodology and Design	10
Test Plan	13
Scope and Depth	14
Schedule and Milestones	15
References	18

Document Control

Document Information

	Information
Document Owner	Jackie Ma
Document Owner ID	A00889988
Issue Date	2025-09-19
Last Saved Date	2026-04-07
File Name	comp8900_jackie_ma_proposal_final_updated.pdf

Document History

Version	Issue Date	Changes
1.0	2025-09-19	Initiation for Proposal Draft 1
2.0	2025-10-26	Add List of Abbreviations and Scope and Depth, update all sections with provided feedback and add references for Proposal Draft 2
3.0	2025-11-09	Update sections with provided feedback, rewrite backend to Flask and add composite indexes for Milestone 1.
4.0	2025-12-02	Update Methodology and Design, Schedule and Milestones section
5.0	2026-02-08	Update proposal to remove temporal correlation logic.
6.0	2026-02-13	Rename 8800 to 8900.
7.0	2026-04-07	Update proposal to reflect completed implementation across milestones 3, 4 and 5.

List of Abbreviations

AIDE - Advanced Intrusion Detection Environment
API - Application Programming Interface
CI - Continuous Integration
FIM - File Integrity Monitoring
HIDS - Host-based Intrusion Detection System
HTML - HyperText Markup Language
HTTP - HyperText Transfer Protocol
HTTPS - HyperText Transfer Protocol Secure
JSON - JavaScript Object Notation
JSONB - JSON Binary (PostgreSQL data type)
OS - Operating System
OSSEC - Open Source HIDS Security
SHA-256 - Secure Hash Algorithm 256-bit
SIEM - Security Information and Event Management
SQL - Structured Query Language
TLS - Transport Layer Security
UI - User Interface

Abstract

Hashiko is a distributed monitoring system that tracks file integrity across multiple endpoints. Agents compute individual SHA-256 hashes for each monitored file and send reports to a central server over HTTPS. This agent-server design enables centralized monitoring across distributed systems. The server stores records and generates alerts when changes are detected. A web interface provides a timeline view of hash history, an alert panel and per-agent configuration management. The system focuses on accuracy, secure transfer and scalability. Automated unit and integration tests cover the agent and server and run on every push via CI. Scope includes agents, server, storage and interface. Out of scope are malware analysis, snapshots and compliance reporting. Hashiko provides centralized visibility into file integrity across distributed systems.

Student Background

I am a fourth-year student in the Bachelor of Applied Computer Science program at BCIT, specializing in Network Security Applications Development. My academic background includes a Computer Systems Technology Diploma from BCIT, specializing in Web and Mobile development. My technical foundation includes proficiency in multiple programming languages: Python, Java, JavaScript, C#, SQL, Swift and Kotlin. I also work with web technologies including HTML and CSS. Frameworks and platform experiences include .NET, Node.js, Express.js, Flask and Laravel. Database experience spans PostgreSQL, MySQL, MariaDB, SQLite and Firebase.

Through previous coursework and personal projects, I have developed several full-stack, backend and database-driven applications. For example, PromptVisioQuiz fetches data from external APIs (CBC RSS feed) and processes it through a multi-service architecture with Node.js/Express and Python/Flask microservices, using JWT authentication and PostgreSQL for data persistence. MealPlanIQ demonstrates my ability to build complex data-driven applications with TypeScript/Angular frontend and Python/Flask backend, managing relational data with PostgreSQL and deploying to cloud infrastructure (GoogleCloud Platform). These projects gave me practical experience in distributed system architecture, RESTful API design, secure authentication, database management and cross-service communication—skills directly applicable to Hashiko's agent-server architecture, event processing pipeline and temporal data storage requirements.

To prepare for this project, I researched file integrity monitoring techniques and distributed security architectures. I read Kaczmarek and Wrobel's analysis of Modern Approaches to File System Integrity Checking approaches to understand current detection methods and their limitations. Their review of user-space and kernel-space integrity checkers helped me understand the trade-offs between periodic checking and real-time monitoring approaches. I also read Abdullah et al.'s work on File Integrity Monitor Scheduling Based on File Security Level Classification scheduling. Their research on combining off-line and on-line monitoring informed my approach to resource prioritization. Beyond these foundational papers, I have

explored existing distributed monitoring tools such as OSSEC and Wazuh to understand agent-server communication patterns. I am currently investigating time-series databases for efficient event storage and message queue systems for reliable data transmission between agents and the central server.

Before studying computer science, I studied architecture and briefly worked in the industry. This background has strengthened my analytical and problem-solving abilities. Examples of my work, such as PromptVisioQuiz and MeanPlanIQ, can be found at www.jackiema.ca. After graduation, I plan to work in software development or IT.

Project Summary

This project is motivated by the growing need for centralized visibility and consistent monitoring in modern distributed systems. As organizations deploy security tools across multiple endpoints, maintaining oversight becomes increasingly difficult. Current security monitoring tools create dangerous blind spots. Traditional file integrity monitors operate independently on each system. They cannot provide unified visibility across the infrastructure. A file modification on one server and a configuration change on another might each seem normal alone. Without centralized monitoring, threats slip through undetected. Organizations need a way to see activity patterns across all systems from a single interface.

Hashiko is a distributed file integrity monitoring system. The project implements a scalable monitoring framework that aggregates hash records from multiple endpoints into a central view. It goes beyond standalone monitoring tools by providing unified visibility across distributed infrastructure. The focus is on centralized collection and reporting. File changes are collected from all monitored systems and stored in one location.

Endpoint agents compute SHA-256 hashes for each file in monitored directories and send the results to a central server. The server stores records and generates alerts when file hashes change between reports. A web interface provides clear visibility, alert tracking and per-agent configuration management. The administrator has a centralized and readable view of file integrity across all monitored machines.

The goal is to enable centralized monitoring that standalone tools cannot provide. By collecting hash records from all endpoints into a unified system, administrators gain complete infrastructure visibility. Expected outcomes include faster breach detection as changes are immediately visible in the central dashboard. False positives decrease because administrators can correlate activity across systems. The system transforms isolated monitoring into unified visibility, providing organizations with actionable insights and improved protection through centralized oversight.

Problem Statement and Background

Organizations face increasing risks from both external attackers and insider threats. Traditional tools such as intrusion detection systems or file integrity monitors can detect single events. Tools like Tripwire and OSSEC use hash comparison to detect file changes [1,2]. AIDE, Samhain and Wazuh also perform file integrity monitoring. AIDE and Samhain use scheduled hash checks. Wazuh extends OSSEC with central logging and rule-based alerts. These tools detect isolated events on individual systems. They do not provide unified visibility across distributed infrastructure. Studies by Abdullah et al. and Kaczmarek and Wrobel confirm that periodic and user-space methods delay detection [1,2]. Their periodic checking creates detection delays and higher inspection frequency improves detection but degrades performance. User-space integrity checkers can only verify if files were modified between checking operations [1]. During this period, attackers can replace system files with backdoors or trojans that may be executed by authorized users [1]. However, they rarely provide centralized management across multiple endpoints. This leaves blind spots where administrators lack visibility into distributed system activity. For example, file changes occurring across multiple servers may seem harmless when viewed in isolation but require centralized oversight for effective monitoring.

The core weakness is the lack of centralized visibility in existing monitoring systems. Unified oversight across endpoints is as important as detecting changes themselves. Traditional FIM tools face a fundamental trade-off. High-frequency inspection maintains effectiveness but hurts performance. Low-frequency inspection reduces overhead but allows threats to persist undetected. Manual policy configuration becomes impractical in distributed environments [2]. Without centralized management, significant risks bypass defences. SIEM tools like Splunk and Elastic Security try to improve visibility. They collect and aggregate logs from multiple sources. However, they focus on log analysis rather than real-time file integrity monitoring. Centralized file integrity monitoring is still missing. Visibility is further reduced by scattered monitoring tools and fragmented dashboards that slow incident response. A centralized system that hash records from multiple endpoints and stores them in one location can close this visibility gap. Centralized monitoring is essential for security. Attackers with administrator privileges can compromise local security tools. Remote monitoring from a privileged server provides protection even when the monitored system is compromised [2]. A web interface adds readability by simplifying alert tracking and management.

Hashiko closes this gap. It uses agents that collect events and send them to a central server. The server stores all activity in one location for unified visibility. This method combines the reach of SIEM tools with the detail of host monitors. It delivers centralized monitoring instead of isolated standalone tools.

Inventiveness

What sets Hashiko apart is its focus on centralized multi-endpoint file integrity monitoring. Most security tools monitor single systems in isolation, such as one server's file changes. The project instead collects data from multiple endpoints into a unified system. It provides visibility that becomes useful only when activity from different systems can be viewed together in one interface. Existing studies focus on improving detection accuracy through frequency tuning and signature updates [1,2]. Few examine centralized collection architectures that combine file integrity monitoring with per-agent configuration management. This unified monitoring approach remains underexplored in host-based intrusion detection research.

Another inventive aspect is the centralized distributed design. Endpoint agents compute per-file hashes and send timestamped reports to the central server. This follows established HIDS deployment patterns but extends beyond traditional file checking. Instead of producing isolated alerts on each endpoint, the system aggregates all records centrally. A web interface presents this data clearly, giving teams both unified oversight and visibility. This blend of distributed data gathering with centralized aggregation is rarely seen in current monitoring solutions. While some commercial SIEM systems such as Splunk Enterprise Security and IBM QRadar provide event aggregation, they focus on log collection rather than file integrity monitoring. Academic systems like I3FS and SOFFIC achieve real-time monitoring but require kernel-level modifications and limited portability. Hashiko's user-space approach provides cross-platform deployment without kernel dependencies.

Traditional FIM solutions require manual policy configuration. Administrators must specify which files to monitor and how often. This becomes impractical in large-scale environments. Hashiko simplifies management through centralized per-agent configuration. Admins can set monitored paths and file size thresholds from the web UI without touching any configuration files on the monitored machine. Unlike systems that monitor only files, Hashiko identifies the exact file that changed rather than reporting a directory-level change. This provides actionable detail that individual tools would miss.

Complexity

The project is complex as the system must collect data from many endpoints and aggregate hash records with precise timestamps. This makes it significantly more advanced than simple standalone monitoring. The system integrates distributed agents, secure real-time communication, centralized hash record storage and a unified web interface. The most difficult aspect is managing secure data collection from multiple distributed endpoints, which is far more challenging than monitoring a single system. Success requires skills in distributed systems, secure communication protocols and scalable database design. This project requires independent design decisions that go beyond coursework. Trade-offs between monitoring frequency and system performance must be experimentally evaluated.

The system requires designing efficient data ingestion pipelines that handle concurrent hash reports from multiple agents. This exceeds traditional FIM approaches. Tools like Tripwire perform simple hash comparisons on individual systems, while centralized monitoring requires handling concurrent data streams from distributed sources. Hashiko requires managing agent authentication, handling network failures gracefully and efficient querying across large datasets. Conventional integrity monitoring tools do not address these distributed system challenges. Implementing secure agent-server communication requires understanding encryption, certificate management and retry logic. No off-the-shelf solution exists for distributed file integrity monitoring with this architecture. The system integrates distributed architecture, per-file hash collection and security monitoring into a cohesive platform. This combination of distributed data collection, secure communication and centralized storage represents Bachelor-level integration.

The web interface presents challenges in state management and alert visualization. The dashboard must display hash records from multiple agents in a unified view, requiring understanding of efficient rendering and change detection across large datasets.

Technical Challenges

Distributed agents must be designed to securely collect and transmit data. Authentication mechanisms are needed to prevent agent spoofing. A one-time registration token exchanges for a persistent API key stored at `/etc/hashiko/api_key` with restricted file permissions. All agent communication uses HTTPS. The agent refuses to connect over plain HTTP. File integrity databases must be secured because if an attacker modifies the database, the entire security system becomes unreliable. Secure agent-server communication is critical, especially when agents operate on potentially compromised systems. Hash data must use encrypted channels to prevent interception.

Network interruptions must be handled without losing hash reports. The agent saves failed reports to a local queue as JSON files and retries them on the next run. Successfully delivered files are removed to prevent duplicates. This queue-based approach ensures data durability without requiring complex message queue systems.

Per-file hashing introduces a performance challenge. Hashing reads the entire file through SHA-256. Directories containing large media files can cause the agent to hang for an extended period. A configurable file size threshold skips files above a set limit. The default is 100 MB and is adjustable per-agent from the web UI. This trade-off between coverage and performance must be carefully managed.

A central server must store and process hash reports from multiple agents. It must handle concurrent data ingestion and efficient querying across large datasets. The storage system uses JSONB in PostgreSQL to store per-file hashes as key-value pairs. This allows flexible schema evolution without column changes. Composite indexes on timestamp and agent ID enable fast queries. The web interface must display thousands of file paths across many agents clearly. The timeline view was designed to show a compact summary per report with expandable detail for changed files.

Methodology and Design

The project follows an agent-collector-storage design with a central server and web interface. This architecture separates data collection from storage and presentation. This pattern appears in multi-platform HIDS deployments. Centralized management enables efficient monitoring of multiple systems. It addresses scalability limitations of standalone FIM tools. Agents compute per-file SHA-256 hashes and send timestamped reports. The server ingests, validates and stores records. The web interface displays hash history and alerts.

System Architecture

Figure 1 illustrates Hashiko's architecture. Endpoints run Linux agents that monitor file changes across configurable directories. Agents authenticate with a Bearer token and transmit hash reports via HTTPS POST to /api/store. The server validates and inserts records into PostgreSQL. A local queue stores failed transmissions for retry on the next run. The web interface requires login and fetches records from /api/hashes. It displays them in a timeline view with change detection and alerts. Figure 2 shows the sequence of operations from agent registration through to the admin view. On the first run, the agent exchanges a registration token for a persistent API key. It then fetches its watch paths and file size threshold from the server. On each subsequent run, the agent walks the monitored directories, computes a SHA-256 hash for each file and submits the report via HTTPS POST. If the server is unreachable, the report is saved to a local queue and retried on the next run. When the admin logs in, the web interface fetches all records from the server, applies change detection and displays the results in a timeline with alerts.

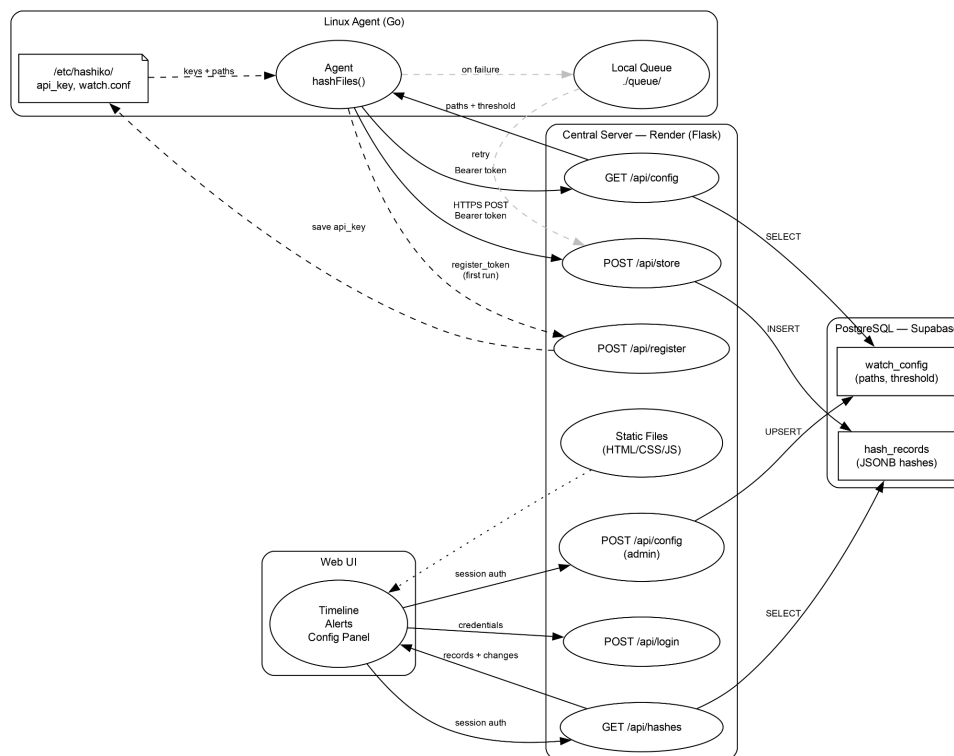


Figure 1 - System Diagram

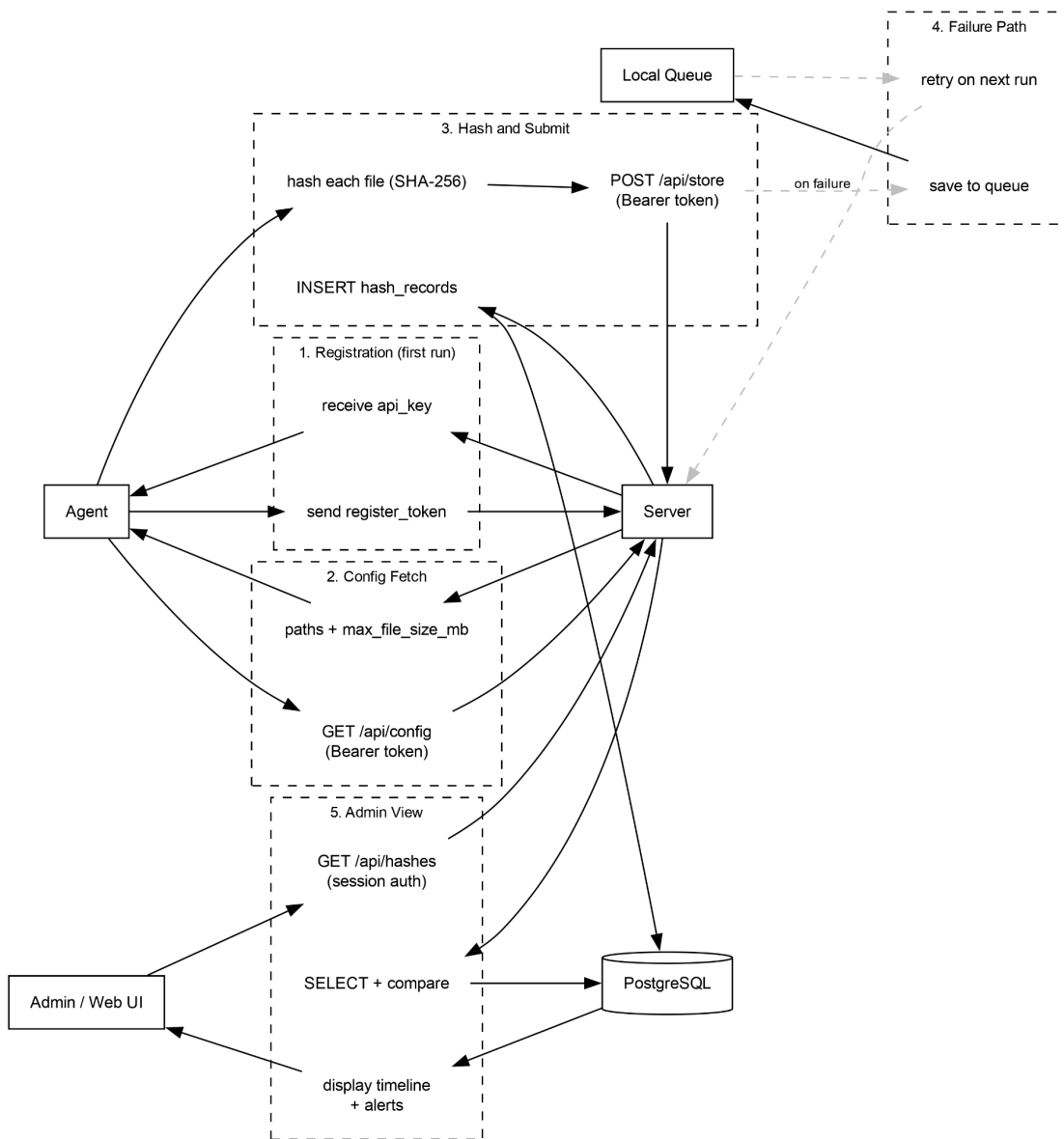


Figure 2 - Sequence Diagram

Agent Architecture

Figure 3 illustrates the agent architecture. The agent uses SHA-256 hashing via `crypto/sha256`. SHA-256 is a cryptographic hash function that produces fixed-size output regardless of input length. This ensures that even a single bit change in a file generates a different hash value. The `hashFiles` function walks each monitored directory and computes an individual SHA-256 hash for every file. Files above the size threshold are skipped. Volatile paths like `.cache`, `.mozilla` and `tmp` are filtered to reduce false positives. Each agent has a unique ID in the format `user@hostname` (machine-id). On the first run, the agent exchanges a registration token for a persistent API key. Subsequent runs load the saved key and fetch watch paths and the size threshold from the server.

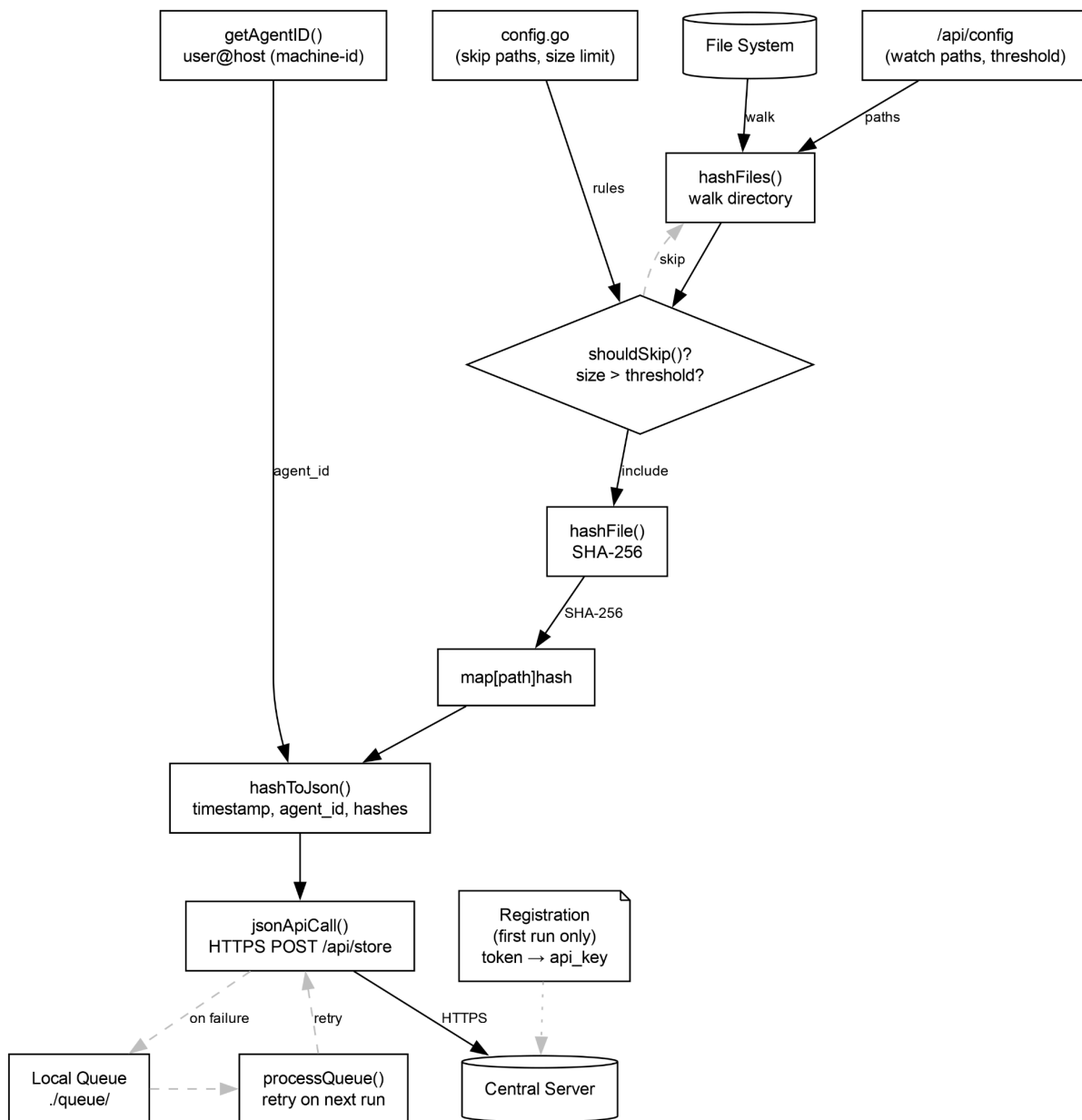


Figure 3 - Agent Diagram

Server Architecture

Built with Flask and PostgreSQL. Loads environment variables and creates a `hash_records` table with `id`, `timestamp`, `agent_id` and a `hashes` JSONB column. A `watch_config` table stores custom watch paths and the file size threshold per agent. The `/api/store` endpoint decodes JSON and inserts records with parameterized queries to prevent SQL injection. The `/api/hashes` endpoint returns all records with change detection applied. The `/api/config` endpoint serves watch paths and threshold to agents. The `/api/register` endpoint exchanges a one-time token for the agent API key. Serves static files on port 8899. Deployed on Render with TLS provided automatically.

Interface

The HTML login page authenticates the admin via session-based auth. Once logged in, the agent grid shows all monitored machines with alert badges. Clicking an agent shows the hash timeline. Each row represents a report with a timestamp, file count and status badge. Rows with changes expand to show the exact files added, modified or deleted. A config panel allows the admin to set watch paths and the file size threshold per agent. An alert panel surfaces all changes detected in the last 24 hours.

Database Schema

Figure 4 shows the database schema. The hash_records table has an auto-incrementing id, TIMESTAMPTZ timestamp, TEXT agent ID and JSONB hashes column. The JSONB column stores a map of file path to SHA-256 hash string. A composite index on (timestamp, agent_id) optimizes queries. The watch_config table stores agent_id, a JSONB array of custom paths and an integer file size threshold. This schema supports any number of monitored paths and files without column changes.

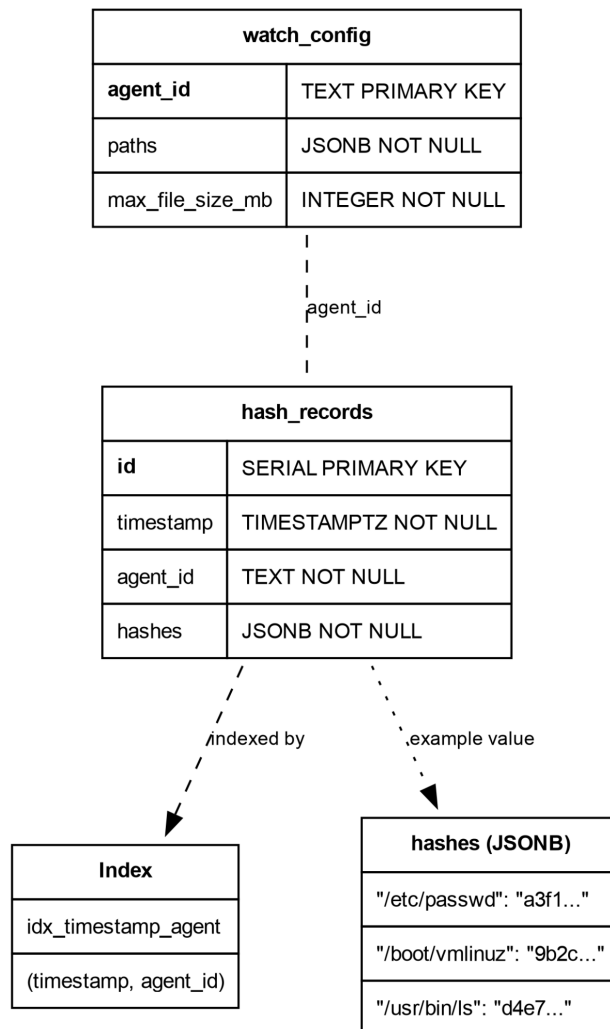


Figure 4 - Database Schema Diagram

Scalability

This modular design enables horizontal scaling. Multiple server instances can process reports concurrently using a shared PostgreSQL database. The JSONB schema supports any number of monitored files per agent without schema changes. Composite indexes enable efficient filtering by time range and agent.

Test Plan

The project is tested using automated unit and integration tests. Tests run automatically on every push via GitHub Actions.

Unit Testing

Go's built-in testing package verifies agent hash computation accuracy, hash stability across runs, change detection, skip path filtering, large file threshold filtering and directory traversal. Python's pytest package verifies the change detection algorithm, all API endpoints, authentication, input validation and config persistence.

Integration Testing

Python tests run against a real PostgreSQL database using Flask's test client. GitHub Actions provides a local Postgres container for CI, keeping tests separate from the production Supabase instance. End-to-end tests verify that hash reports are stored correctly and that change detection identifies modified files accurately.

Performance Testing

The load test runs 10 agents every 30 seconds for 10 minutes. Measures response time (target p95 under 200ms), throughput (over 20 per second) and memory (under 100MB). Stress test scales to 100 agents to find the breaking point. Query test measures response time with 1000 to 100,000 records (target under 500ms).

System Testing

File integrity validation includes monitoring /etc/passwd for unauthorized modifications. Modification by any process should trigger an alert. Unauthorized access simulation includes detecting changes to system binaries in /usr/bin and /usr/sbin. Mass modification simulation includes 20 rapid file changes across monitored directories.

Test Environment

Main testing on desktop with Fedora 42, Python 3.14 and Go 1.24.8.

Success Metrics

Over 80% code coverage, under 1% integration errors, zero false negatives on file integrity monitoring, under 5% false positives and performance targets met. Unit tests run first, followed by integration tests if units pass. Failed tests block merges. Weekly system and performance tests.

Expected Outcomes

- The system detects unauthorized file modifications.
- Alerts are generated when file hashes change between reports.
- No reports are lost during network interruptions.
- All significant test cases result in measurable logs and clear status.
- False positives are minimized during normal usage.

Sample Test Cases

- Detect file modification: modify a file in /usr/bin on a monitored agent. Expect the system to detect the hash change and generate an alert.
- Replay normal activity: write benign files to monitored directories. Expect zero alerts.
- Network failure simulation: disconnect agent network during reporting, then reconnect. Expect reports are queued and delivered after reconnection with no lost or duplicate records.
- Multi-agent monitoring: generate file modification events from three separate agents simultaneously. Expect all events to be received, stored and displayed correctly in the web interface.

Scope and Depth

Scope includes agents, a central server, storage and a web interface. Features cover per-file integrity monitoring with SHA-256 hashes, timestamped reports, HTTPS-based transfer, queue-based retry logic, alerts, per-agent watch path configuration, file size threshold configuration and a timeline visualization. Deliverables are a Go agent, Flask server, PostgreSQL storage, web interface, documented APIs and a test suite with CI.

Out of scope are process monitoring, login monitoring, memory scanning, kernel exploit detection, malware analysis and compliance reports. These exclusions distinguish Hashiko from comprehensive HIDS platforms like OSSEC. OSSEC includes rootkit detection and compliance reporting. Hashiko focuses on centralized per-file hash collection and storage.

Depth comes from per-file hash precision, centralized aggregation, durable storage with retry capability, configurable alerts, unified dashboards, per-agent configuration management and full security for identity, encryption and session management. This depth differentiates Hashiko from traditional FIM. Tools like Samhain and AIDE provide centralized management and periodic checking but lack per-file precision and unified web interfaces. Some advanced systems like I3FS and SOFFIC operate at the kernel level to provide real-time file checking. However, these require kernel modifications and are platform-specific. Hashiko takes a different approach, using

user-space agents with centralized storage rather than kernel integration. Hashiko addresses the performance versus effectiveness trade-off by skipping large files and filtering volatile paths rather than attempting continuous file hashing.

Schedule and Milestones

This project spans two academic terms. It covers both COMP 8800 and COMP 8900. The following tables show a detailed week-by-week schedule. The plan outlines all key project milestones. This includes research, implementation and testing.

COMP 8800 focuses on core features. The goal is a functional prototype. It also includes the finalized project proposal. COMP 8900 builds advanced features. This term focuses on hardening the system. It concludes with the final project submission and presentation. All deliverables align with course milestones.

COMP 8800		
Week	Key Activities & Focus	Due Date / Milestone
1	Formulate project idea. Draft sections: Student Background, Project Summary.	
2	Draft sections: Inventiveness, Complexity, Technical Challenges.	
3	Draft Problem Statement. Revise Project Summary. Compile sections 1-6.	Proposal Draft 1 Due
4	Build initial prototype: agent hashing, basic server endpoint for data ingest.	
5	Refine prototype. Draft Methodology and Test Plan sections based on findings.	
6	Complete and document prototype for submission. Draft Scope, Schedule and References.	Prototype Due
7	Revise all proposal sections, incorporating insights from the prototype.	Proposal Draft 2 Due
8	Address initial feedback on Draft 2. Begin Milestone 1: Implement composite index and rewrite backend to Flask.	
9	Continue development on Milestone 1 deliverables.	
10	Complete all coding and documentation for Milestone 1.	Milestone 1 Due
11	Review feedback from instructor and second reader. Plan proposal revisions.	
12	Revise proposal based on all feedback. Begin Milestone 2: Implement message queue and retry logic.	
13	Continue development on Milestone 2 deliverables.	
14	Continue Milestone 2. Address proposal feedback.	
15	Integrate all Milestone 2 components. Complete Final Proposal, demo and give a presentation.	Final Proposal & Milestone 2 Due

COMP 8900		
Week	Key Activities & Focus	Due Date / Milestone
1 - 5	Begin Milestone 3: Implement hash comparison, login and alert system.	Milestone 3 Due
6 - 10	Begin Milestone 4: Implement agent registration, per-agent watch path configuration, unique agent name, security hardening and deployment with TLS.	Milestone 4 Due
11 - 14	Begin Milestone 5: Implement per-file hashing, file size threshold and automated tests.	Milestone 5 Due
15	Deliver final presentation. Submit all project files.	Final Project Submission

References

- [1] J. Kaczmarek and M. Wrobel, "Modern Approaches to File System Integrity Checking," *2008 1st International Conference on Information Technology*, pp. 1–4, May 2008. doi:10.1109/inftech.2008.4621669
- [2] Z. H. Abdullah, N. I. Udzir, R. Mahmud and K. Samsudin, "File Integrity Monitor Scheduling Based on File Security Level Classification," *Communications in Computer and Information Science*, pp. 177–189, 2011. doi:10.1007/978-3-642-22191-0_16